

HYPERSKILL: Self-Evolving LLM Agents via Hypergraph-Structured Skill Memory

Ruiyao Xu¹ Tiankai Yang² Wei-Chieh Huang³

¹Northwestern University

²University of Southern California

³University of Illinois at Chicago

Abstract

As agentic tasks grow in complexity, LLM agents increasingly rely on experiential memory to reuse procedural knowledge across tasks. Effective memory design must jointly address *what to store*, *how is structured and retrieved*, and *how memory evolves*. Existing systems tackle each only partially: they store trajectories, insights, or workflows as isolated entries, discarding compositional relationships among subtasks and reusable skills; retrieve by flat embedding similarity that ignores relational signals; and maintain memory without leveraging its relational structure. We propose HYPERSKILL, a hypergraph-based memory framework that jointly improves all three. HYPERSKILL represents memory as a hypergraph with two node types, subtask steps and reusable skills, where each hyper-edge links the subtasks and skills from a single trajectory. Dual-path retrieval queries both subtask and trajectory levels, ranking skills by co-occurrence across retrieved trajectories. Periodic structure-informed maintenance prunes low-utility nodes and merges redundant skills via quality-weighted propagation. Across xBench, GAIA, and WebWalkerQA with GPT-4o and Qwen3-30B-A3B, HYPERSKILL outperforms ten memory baselines, yielding gains of up to +11.51 on GAIA and +11.18 on WebWalkerQA. ¹

1 Introduction

The rapid development of LLM agents (Liu et al., 2025b; Yao et al., 2022; Shinn et al., 2023) has significantly broadened the scope of tasks that autonomous systems can tackle, spanning long-horizon web navigation (Chen et al., 2025; Gur et al., 2024; Mialon et al., 2023), interactive computer use (Yang et al., 2024; Xie et al., 2024), and multi-step scientific reasoning (Ghafarollahi and Buehler, 2025; Ren et al., 2025). As tasks grow in

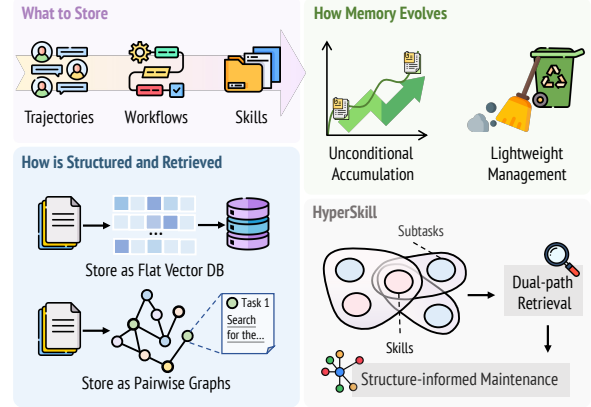


Figure 1: Three design dimensions of experiential agent memory. HYPERSKILL unifies *what to store*, *how memory is structured and retrieved*, and *how memory evolves* via hyperedges over subtasks and skills, dual-path retrieval, and structure-informed maintenance.

complexity and agents operate across a continuous stream of episodes, a single interaction is rarely sufficient to solve a problem reliably. Agent memory systems (Huang et al., 2026; Hu et al., 2025) address this by storing and retrieving information beyond a single context window. Among them, *experiential memory* (Wang et al., 2025b; Ouyang et al., 2026) is uniquely important: rather than encoding external facts, it distills the agent’s own past trajectories into reusable knowledge, enabling *self-evolving* (Gao et al., 2025; Lopez-Paz and Ranzato, 2017) agents that grow more competent with each trajectory instead of starting from scratch.

This has motivated a growing line of research on self-evolving agent memory (Ouyang et al., 2026; Wang et al., 2025b; Tang et al., 2025b; Zhang et al., 2025b). Early work stored raw interaction trajectories as flat records (Wang et al., 2023a; Zheng et al., 2024; Zhao et al., 2024), while subsequent approaches shifted toward distilling experience into more compact, transferable units such as workflows (Wang et al., 2025b), reasoning strategies (Ouyang et al., 2026), and execution skills (Xia et al., 2026). Despite this progress, the design of ex-

¹Code is available at [\[link\]](#)

periential memory systems remains an open problem that can be decomposed into three fundamental questions: *what to store*, *how is structured and retrieved*, and *how memory evolves*. Existing systems address each only partially, leaving significant gaps that limit their capacity for self-improvement.

On *what to store*, early systems retain raw trajectories or distill them into insights and workflows (Zhao et al., 2024; Wang et al., 2023a, 2025b), while recent work abstracts experience into reusable skills (Ouyang et al., 2026; Zheng et al., 2025; Anthropic, 2024). However, these systems discard the compositional relationships among subtasks and skills. Since tasks are inherently compositional, retrieval cannot leverage shared procedural structure across superficially different tasks when these relationships are not retained. On *how memory is structured and retrieved*, most approaches adopt flat vector stores or simple binary graphs (Zhang et al., 2025a; Chhikara et al., 2025; Rasmussen et al., 2025) and retrieve by embedding similarity alone, which cannot represent an n-ary co-occurrence between a skill and the subtask patterns it has repeatedly succeeded with. On *how memory evolves*, the majority of systems accumulate knowledge without any curation (Wang et al., 2023a; Zhao et al., 2024), and the few that do introduce only lightweight operations such as pruning or deduplication (Fang et al., 2026; Zheng et al., 2025), applied independently to each node.

These gaps share a common *structural root*: a trajectory is not a single unit of knowledge but an n -ary association among subtasks, skills, and an outcome, structure that flat lists and pairwise edges represent only lossily. Hypergraphs (Zhou et al., 2006; Tang et al., 2025a) preserve such associations natively by letting each hyperedge connect more than two nodes simultaneously. We propose HYPERSKILL, which organizes experiential memory as a hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with subtask nodes $\mathcal{V}_u$ and skill nodes $\mathcal{V}_s$, where each hyperedge groups the subtasks and skills from a single trajectory together with a distilled lesson and a utility score. This design directly improves the three gaps: ❶ hyperedges preserve the full compositional context that flat stores discard; ❷ skill nodes are shared across hyperedges, enabling us to rank a candidate skill by how frequently it appears across the retrieved trajectories, a structural signal that semantic similarity alone cannot provide; and ❸ quality-weighted propagation over the hypergraph enables merging redundant skills by jointly

considering node utility and neighborhood topology. Across three agentic benchmarks (xBench, GAIA, and WebWalkerQA) under two different model families, HYPERSKILL consistently outperforms ten memory baselines.

2 Related Work

Memory Mechanisms in Agent Systems. LLM agents are increasingly equipped with external memory to overcome fixed context windows (Huang et al., 2026), serving roles such as factual memory (Gutiérrez et al., 2024; Rasmussen et al., 2025), working memory (Packer et al., 2023), and experiential memory, which distills the agent’s own trajectories into reusable knowledge for self-improvement. We focus on the latter and categorize prior work along three dimensions, identifying a gap in each. (i) *What is stored*: Early works retain raw trajectories (Zheng et al., 2024; Wen et al., 2024), which preserve context but carry noise; later approaches abstract trajectories into insights or workflows (Zhao et al., 2024; Ouyang et al., 2026; Wang et al., 2025b; Fang et al., 2026), gaining generality at the cost of procedural detail; recent work distills reusable skills (Zheng et al., 2025; Wang et al., 2023a; Xia et al., 2026) but treats each skill in isolation. None preserves reusable subtask decompositions, limiting transfer of compositional planning across episodes. (ii) *How memory is structured and retrieved*: Most systems use flat vector or JSON stores with embedding-based retrieval (Zhao et al., 2024; Ouyang et al., 2026; Wang et al., 2025b; Zheng et al., 2025); a few adopt graph structures (Zhang et al., 2025a; Yang et al., 2026) to capture relational dependencies, but pairwise edges decompose an n-ary co-occurrence into disconnected binary links and discard the joint compositional context of an episode. (iii) *How memory evolves*: Most systems accumulate unconditionally (Wang et al., 2023a; Zhao et al., 2024; Wang et al., 2025b); a few add lightweight per-entry pruning (Zheng et al., 2025), consolidation (Zhang et al., 2025a), or deduplication (Tang et al., 2025b), but none leverages structural signals or jointly considers quality and graph topology. A more detailed discussion is in Appendix E.

and retrieved, and *how* memory evolves over time, identifying a gap in each. (i) *What is stored*: Early works store raw trajectories of entire episodes (Zheng et al., 2024; Wen et al., 2024), which preserve full context but include substan-

tial noise; later approaches abstract experience into insights and workflows (Zhao et al., 2024; Ouyang et al., 2026; Wang et al., 2025b; Fang et al., 2026), improving generalizability but discarding fine-grained procedural detail; while others distil reusable skills from trajectories (Zheng et al., 2025; Wang et al., 2023a; Xia et al., 2026), retaining concise knowledge units but treating each skill in isolation without compositional structure. Existing systems don’t maintain reusable subtask decompositions that capture what procedural sequence was used to solve a task, limiting the agent’s ability to transfer compositional planning knowledge across episodes. (ii) *How memory is structured and retrieved*: Most systems store memory as flat vector databases or JSON files and rely solely on semantic similarity for retrieval (Zhao et al., 2024; Ouyang et al., 2026; Wang et al., 2025b; Zheng et al., 2025), limiting both the relevance and comprehensiveness of retrieved knowledge. A few works explore graph structures to capture relational dependencies (Zhang et al., 2025a; Yang et al., 2026; Xu and Ding, 2026), enabling structural traversal, but rely on pairwise edges that decompose n -ary co-occurrence into disconnected binary links and discard the joint compositional context of an episode. (iii) *How memory evolves*: Most systems accumulate memory unconditionally (Wang et al., 2023a; Zhao et al., 2024; Wang et al., 2025b); a few add lightweight management such as pruning (Zheng et al., 2025), consolidation (Zhang et al., 2025a), or deduplication (Tang et al., 2025b), but all operate on individual entries without leveraging structural information, with no maintenance guided jointly by quality signals and graph topology. A more detailed discussion is provided in Appendix E.

3 HYPERSKILL

3.1 Preliminaries

Agent, Environment, and Memory. Following previous work (Zhang et al., 2025b), we formalize an LLM-based agent system as a policy π_θ interacting with an environment over trajectories $\tau_i = (d_i, o_i, a_i, r_i)$, where d_i is a task description, o_i and a_i are observation and action sequences, and $r_i \in \{\text{success}, \text{fail}\}$ is the outcome. At each step t , the agent selects $a_t \sim \pi_\theta(\cdot \mid o_t, m_t)$, where m_t is a memory context retrieved from an external memory module $\mathcal{M}$. Given growing history $\mathcal{H} = \{\tau_1, \dots, \tau_N\}$, $\mathcal{M}$ supports retrieval $m_t = g_{\mathcal{M}}(d, \mathcal{H})$ and update $\mathcal{M} \leftarrow \text{UPDATE}(\mathcal{M}, \tau_i)$,

with the objective of maximizing expected task performance:

$$\max_{\mathcal{M}} \mathbb{E}_{d \sim \mathcal{D}} [R(\pi_\theta(\cdot \mid d, g_{\mathcal{M}}(d, \mathcal{H})))] . \quad (1)$$

3.2 HYPERSKILL Memory Architecture

A hypergraph (Zhou et al., 2006) $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ generalizes ordinary graphs by allowing each hyper-edge $e \in \mathcal{E} \subseteq 2^{\mathcal{V}}$ to connect an arbitrary subset of nodes ($|e| \geq 2$), capturing higher-order n -ary relationships among multiple nodes that pairwise edges cannot represent. HYPERSKILL exploits this structure to encode both compositional task decompositions and reusable execution knowledge in a single unified memory.

[>] **Nodes.** Each $v_i \in \mathcal{V}$ is a discrete, reusable unit of task knowledge distilled from agent experience. The node set partitions as $\mathcal{V} = \mathcal{V}_u \cup \mathcal{V}_s$, where each node can be represented by a tuple:

$$v_i \triangleq (c_i, \ell_i, \gamma_i), \quad \ell_i \in \{u, s\}, \quad (2)$$

with c_i as natural-language content, ℓ_i as type label, and $\gamma_i \in [0, 1]$ as *utility score*, a dynamic measure that tracks empirical success rate and step efficiency. Concretely, for all nodes and hyperedges that participated in retrieval during trajectory τ_i , we increment $\sigma(\cdot)$ on success and update $\bar{T}(\cdot)$ with step count T_i , then recompute:

$$\gamma(\cdot) = \beta \cdot \frac{\sigma(\cdot)}{\nu(\cdot)} + (1 - \beta) \cdot \left(1 - \frac{\bar{T}(\cdot) - T_{\min}}{T_{\max} - T_{\min}}\right), \quad (3)$$

where $\nu(\cdot)$ is the total retrieval count, $\sigma(\cdot)$ the success count, $\bar{T}(\cdot)$ the running average step count, and β balances success rate against step efficiency. The two node types reflect complementary sides of task knowledge: *subtask nodes* encode the compositional workflow structure of a task, while *skill nodes* encode reusable execution strategies that transfer across tasks.

- **Subtask nodes** $\mathcal{V}_u$: each $v_u \triangleq (c_u, u, \gamma_u)$ captures one step in a task decomposition. The content c_u encodes a step description together with any preconditions under which it applies.
- **Skill nodes** $\mathcal{V}_s$: each $v_s \triangleq (c_s, s, \gamma_s)$ captures reusable knowledge extracted via outcome-conditioned prompting. The content c_s encodes a concise strategy label and a description covering what to do, when to apply it, and why it works (or fails). Successful trajectories yield *positive strategies* to follow; failed trajectories yield *error patterns* to avoid.

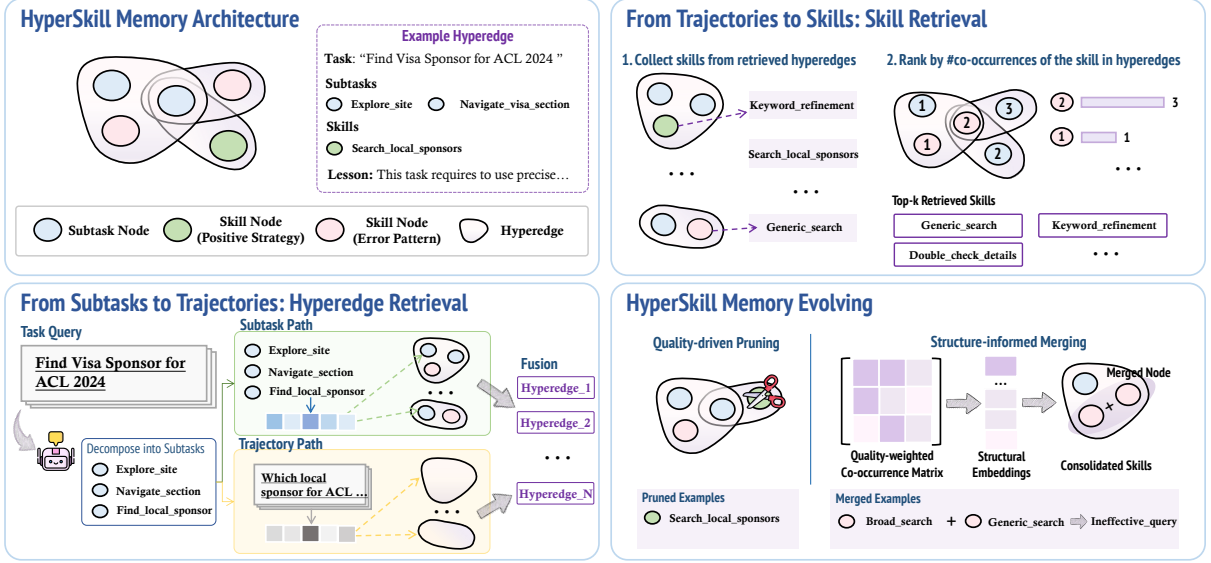


Figure 2: Overview of HYPERSKILL. Given a new task, the agent decomposes it into subtasks, retrieves relevant trajectories via dual-path hyperedge retrieval, ranks skills by co-occurrence, and executes with the retrieved context. After task completion, new nodes and a hyperedge are added to the memory, which periodically undergoes pruning and merging.

[>] **Hyperedges.** Each hyperedge $e_j \in \mathcal{E}$ corresponds to a single trajectory τ_j and is defined as a tuple $e_j \triangleq (V_j, d_j, \ell_j, \gamma_j)$, where $V_j = V_j^u \cup V_j^s$, with $V_j^u \subseteq \mathcal{V}_u$ the subtask nodes and $V_j^s \subseteq \mathcal{V}_s$ the skill nodes extracted from τ_j ; d_j is the original task description; and ℓ_j is a concise *distilled lesson* capturing the transferable pattern the trajectory reveals. For retrieval, each hyperedge is represented by the embedding $\mathbf{h}_e = \phi(d_j \parallel \ell_j)$, which concatenates the task description with the distilled lesson so that similarity matching captures both surface wording and the underlying transferable pattern.

3.3 From Subtasks to Trajectories: Hyperedge Retrieval

Prior methods explore diverse forms of experiential knowledge, from raw trajectories (Zheng et al., 2024; Wen et al., 2024) and distilled insights (Ouyang et al., 2026; Zhao et al., 2024) to reusable skills (Zheng et al., 2025; Wang et al., 2023a), but most rely on the full task description as the sole retrieval query. This misses trajectories that are superficially different yet share similar procedural subtask patterns. To retrieve more comprehensive trajectory-level context, HYPERSKILL queries the hypergraph through two complementary paths: one decomposes the current task into subtasks and matches them against stored subtask nodes, surfacing trajectories with shared procedural structure regardless of task-level wording; the other matches the task description directly against hyper-

edge representations that encode both the original task and its distilled lesson.

Task decomposition and subtask path. Upon receiving a new task d_q , the agent first invokes an LLM call to produce a fine-grained decomposition $\mathcal{P}_0 = \{d_1, \dots, d_m\}$, where each d_i describes a subtask unit. This decomposition mirrors the subtask structure already stored in the hypergraph, enabling retrieval at the subtask level rather than the full-task level. We encode the full decomposition as a single vector $\mathbf{h}_{\mathcal{P}_0} = \phi(\mathcal{P}_0)$ using a fixed-length text encoder ϕ , and retrieve the top- k_u subtask nodes whose embeddings $\mathbf{h}_v = \phi(c_v)$ are most similar:

$$R_u = \text{top-}k_u \sim_{v \in \mathcal{V}_u} (\mathbf{h}_{\mathcal{P}_0}, \mathbf{h}_v). \quad (4)$$

Because each subtask node is incident to one or more hyperedges, the matched nodes naturally expand to a set of candidate trajectories: $\mathcal{E}_{\text{sub}} = \{e \in \mathcal{E} \mid V_e \cap R_u \neq \emptyset\}$, where V_e denotes the node set of hyperedge e . This path is particularly effective at surfacing trajectories that share procedural substructure with the current task, even when their task-level descriptions differ substantially.

Trajectory path and fusion. The subtask path alone may miss trajectories whose overall task descriptions are highly relevant but whose individual subtask nodes were not among the top- k_u matches. A complementary trajectory-level path addresses this by encoding the full task description as $\mathbf{h}_{d_q} = \phi(d_q)$ and ranking hyperedges directly

by similarity to the query:

$$\mathcal{E}_{\text{traj}} = \text{top-}k_e \text{ sim}(\mathbf{h}_{d_q}, \mathbf{h}_e), \quad (5)$$

where $\mathbf{h}_e = \phi(d_e \parallel \ell_e)$ concatenates the original task description with the distilled lesson, so that ranking captures both surface wording and the underlying transferable pattern. Candidates from both paths are merged into a unified trajectory set:

$$\mathcal{E}^* = \mathcal{E}_{\text{sub}} \cup \mathcal{E}_{\text{traj}}. \quad (6)$$

Together, the two paths are complementary: the subtask path retrieves trajectories with shared procedural substructure, while the trajectory path retrieves those with similar high-level task semantics. The distilled lessons $\mathcal{L}_q = \{\ell_e \mid e \in \mathcal{E}^*\}$ from the fused set are provided to the agent as trajectory-level context during execution.

3.4 From Trajectories to Skills: Skill Retrieval

Given the fused trajectory set $\mathcal{E}^*$, the next step is to select which skills to surface as execution guidance. Each hyperedge in $\mathcal{E}^*$ contains the skill nodes extracted from that trajectory, so the candidate pool is their union:

$$R_s = \bigcup_{e \in \mathcal{E}^*} V_e^s \subseteq \mathcal{V}_s, \quad (7)$$

where V_e^s denotes the skill nodes of hyperedge e . A flat memory system would rank these candidates by embedding similarity to d_q alone. However, semantic similarity only measures whether a skill’s description *looks like* the current task; it does not capture whether the skill has actually been applied in trajectories relevant to the current task.

Co-occurrence ranking. HYPERSKILL exploits the fact that skill nodes, unlike subtask nodes, are deduplicated and reused across trajectories. A skill that appears in many of the retrieved trajectories has been repeatedly applied in task-relevant contexts and is therefore more likely to be useful. We formalize this with a simple co-occurrence count: for each candidate $v \in R_s$, we count how many retrieved hyperedges contain v :

$$\kappa(v) = |\{e \in \mathcal{E}^* \mid v \in V_e\}|. \quad (8)$$

The top- k_s skills by κ are selected as execution guidance, with ties broken by embedding similarity to d_q :

$$\mathcal{S}_q = \text{top-}k_s \kappa(v). \quad (9)$$

Execution and memory update. The retrieved context $(\mathcal{S}_q, \mathcal{L}_q)$ is assembled once per task and held fixed throughout execution, instantiating the memory context $m_t = (\mathcal{S}_q, \mathcal{L}_q)$. At each step t , the agent selects an action conditioned on the current observation o_t , the retrieved skills $\mathcal{S}_q$, the distilled lessons $\mathcal{L}_q$, and a sliding window of recent interaction history:

$$a_t \sim \pi_\theta(\cdot \mid o_t, \mathcal{S}_q, \mathcal{L}_q, \text{hist}_{t-w:t-1}). \quad (10)$$

Upon task completion with outcome r_i and step count T_i , an outcome-conditioned LLM call extracts skill nodes S_i^{new} and a distilled lesson ℓ_{e_i} from the full trajectory. Subtask nodes U_i^{new} are taken directly from $\mathcal{P}_0$. Before inserting each newly extracted skill node into $\mathcal{V}_s$, we compute its embedding similarity to all existing skill nodes; if the similarity exceeds a threshold δ_{dedup} , the new node is merged into its nearest existing neighbor rather than added as a duplicate, keeping the skill vocabulary compact and reusable. A new hyperedge over $U_i^{\text{new}} \cup S_i^{\text{new}}$ is then added to $\mathcal{E}$ with lesson ℓ_{e_i} and task description d_q ; its embedding is $\phi(d_q \parallel \ell_{e_i})$, matching Eq. 5. The utility score $\gamma(\cdot)$ is recomputed via Eq. 3 for all elements that participated in retrieval.

3.5 HYPERSKILL Memory Evolving

As HYPERSKILL accumulates tasks, $\mathcal{G}$ risks growing stale: nodes may become semantically redundant, and repeatedly retrieved but low-quality knowledge can actively mislead future decisions. To keep memory compact, non-redundant, and high-quality, HYPERSKILL periodically refines $\mathcal{G}$ every N_{maint} tasks via two operations: *quality-driven pruning* and *structure-informed merging*.

Quality-driven pruning. Nodes that have been retrieved sufficiently often but consistently fail to improve agent outcomes are explicitly removed from $\mathcal{G}$:

$$\mathcal{V} \leftarrow \mathcal{V} \setminus \{v \in \mathcal{V} \mid \nu(v) \geq N_{\text{min}} \wedge \gamma(v) < \tau_{\text{prune}}\}, \quad (11)$$

where $\nu(v)$ is the total number of times v has been retrieved and $\gamma(v)$ is its composite quality score (Eq. 3), reflecting empirical success rate and step efficiency accumulated over those retrievals. When a node is deleted, it is removed from all hyperedge memberships.

Structure-informed merging. Skill nodes that exhibit strong quality-weighted co-occurrence *and* encode semantically similar execution knowledge are

Table 1: Main results across three benchmarks. We report success rate (SR %), average number of steps (#Steps), and average tool calls (#Calls) per task. **Best**; 2nd best per backbone. $\uparrow/\downarrow$: change vs. No Memory. For #Steps and #Calls, lower is better.

Backbone	Method	xBench			GAIA			WebWalkerQA		
		SR	#Steps	#Calls	SR	#Steps	#Calls	SR	#Steps	#Calls
 Qwen3-30B-A3B	No Memory	46.00 _{0.00}	4.87 _{0.00}	5.05 _{0.00}	32.12 _{0.00}	8.23 _{0.00}	4.75 _{0.00}	39.41 _{0.00}	4.07 _{0.00}	3.87 _{0.00}
	ReasoningBank	43.00 _{0.30}	4.47 _{0.40}	4.72 _{0.33}	29.70 _{0.42}	7.44 _{0.79}	4.42 _{0.33}	44.71 _{0.30}	3.64 _{0.43}	3.72 _{0.15}
	ExpeL	45.00 _{0.10}	5.14 _{0.27}	6.06 _{0.10}	33.33 _{0.21}	7.38 _{0.85}	4.15 _{0.60}	47.06 _{0.65}	4.04 _{0.03}	3.94 _{0.07}
	AWM	44.00 _{0.20}	4.86 _{0.01}	5.49 _{0.44}	32.73 _{0.61}	7.07 _{0.16}	3.92 _{0.83}	41.76 _{0.35}	3.84 _{0.23}	3.72 _{0.15}
	Generative	49.00 _{0.30}	4.46 _{0.41}	4.97 _{0.08}	34.55 _{0.43}	7.43 _{0.80}	3.97 _{0.78}	41.18 _{0.77}	3.82 _{0.25}	3.87 _{0.00}
	Voyager	50.00 _{0.40}	4.77 _{0.10}	5.05 _{0.00}	34.55 _{0.43}	7.47 _{0.76}	4.16 _{0.59}	46.47 _{0.06}	3.84 _{0.23}	3.82 _{0.05}
	DILU	52.00 _{0.60}	5.01 _{0.14}	5.45 _{0.40}	32.12 _{0.00}	7.14 _{0.09}	4.17 _{0.58}	44.12 _{0.71}	3.87 _{0.20}	3.95 _{0.08}
	Cheatsheet	37.00 _{0.90}	4.03 _{0.84}	4.84 _{0.21}	27.88 _{0.24}	6.84 _{0.39}	5.28 _{0.53}	39.41 _{0.00}	3.62 _{0.45}	4.06 _{0.19}
	MemP	45.00 _{0.10}	4.42 _{0.45}	4.24 _{0.81}	32.73 _{0.61}	6.97 _{0.26}	4.42 _{0.33}	45.88 _{0.47}	3.72 _{0.35}	3.66 _{0.21}
	MemEvolve	39.00 _{0.70}	4.64 _{0.23}	4.72 _{0.33}	28.48 _{0.64}	5.08 _{0.15}	3.67 _{0.08}	37.06 _{0.35}	3.61 _{0.46}	3.32 _{0.55}
	PlugMem	42.00 _{0.40}	5.07 _{0.20}	5.18 _{0.13}	34.55 _{0.43}	7.02 _{0.21}	4.18 _{0.57}	35.88 _{0.53}	3.96 _{0.11}	3.83 _{0.04}
 GPT-4o	HYPERSKILL	52.00 _{0.60}	4.52 _{0.35}	4.88 _{0.17}	36.97 _{0.85}	7.25 _{0.98}	5.48 _{0.73}	50.59 _{0.18}	4.04 _{0.03}	4.06 _{0.19}
	No Memory	54.00 _{0.00}	6.29 _{0.00}	6.33 _{0.00}	32.73 _{0.00}	6.59 _{0.00}	8.23 _{0.00}	46.47 _{0.00}	5.82 _{0.00}	7.04 _{0.00}
	ReasoningBank	57.00 _{0.30}	8.22 _{0.93}	12.27 _{0.94}	33.33 _{0.60}	7.21 _{0.62}	10.17 _{0.94}	47.65 _{0.18}	5.59 _{0.23}	6.45 _{0.59}
	ExpeL	54.00 _{0.00}	5.27 _{0.02}	6.11 _{0.22}	35.76 _{0.03}	6.63 _{0.04}	9.48 _{0.25}	47.06 _{0.59}	5.58 _{0.24}	6.32 _{0.72}
	AWM	52.00 _{0.20}	6.47 _{0.18}	8.12 _{0.79}	35.15 _{0.42}	7.13 _{0.54}	9.08 _{0.85}	47.65 _{0.18}	5.77 _{0.05}	6.49 _{0.55}
	Generative	56.00 _{0.20}	7.65 _{0.36}	11.34 _{0.01}	36.36 _{0.63}	6.39 _{0.20}	7.68 _{0.55}	51.18 _{0.71}	5.77 _{0.05}	6.33 _{0.71}
	Voyager	47.00 _{0.70}	7.99 _{0.70}	11.27 _{0.94}	31.52 _{0.21}	6.52 _{0.07}	9.15 _{0.92}	48.82 _{0.35}	6.07 _{0.25}	6.93 _{0.11}
	DILU	53.00 _{0.10}	7.16 _{0.87}	9.98 _{0.65}	33.33 _{0.60}	5.34 _{0.25}	5.63 _{0.60}	48.24 _{0.77}	6.05 _{0.23}	6.97 _{0.07}
	Cheatsheet	55.00 _{0.10}	7.56 _{0.27}	10.92 _{0.59}	35.15 _{0.42}	6.27 _{0.32}	8.36 _{0.13}	47.06 _{0.59}	6.02 _{0.20}	7.25 _{0.21}
	MemP	48.00 _{0.60}	7.59 _{0.30}	10.76 _{0.43}	33.33 _{0.60}	7.04 _{0.45}	8.64 _{0.41}	50.59 _{0.12}	5.66 _{0.16}	6.03 _{0.01}
 GPT-4o	MemEvolve	47.00 _{0.70}	6.68 _{0.39}	9.06 _{0.73}	36.36 _{0.63}	6.97 _{0.38}	7.29 _{0.94}	48.24 _{0.77}	6.09 _{0.27}	7.34 _{0.30}
	PlugMem	59.00 _{0.50}	5.94 _{0.35}	6.22 _{0.11}	41.94 _{0.21}	5.73 _{0.86}	5.11 _{0.12}	47.06 _{0.59}	5.96 _{0.14}	6.49 _{0.55}
	HYPERSKILL	62.00 _{0.80}	5.27 _{0.02}	6.52 _{0.19}	44.24 _{0.51}	6.01 _{0.58}	7.35 _{0.88}	51.18 _{0.71}	5.93 _{0.11}	7.04 _{0.00}

candidates for consolidation into a single higher-order skill. Subtask nodes $\mathcal{V}_u$ are excluded, as merging planning steps would corrupt the compositional structure of task decompositions. To identify merge candidates within $\mathcal{V}_s$, we compute *quality-weighted structural embeddings*. We first construct a co-occurrence matrix $\mathbf{W} \in \mathbb{R}^{|\mathcal{V}_s| \times |\mathcal{V}_s|}$:

$$W_{ij} = \min(\gamma(v_i), \gamma(v_j)) \cdot \sum_{\substack{e \in \mathcal{E} \\ v_i \in V_e, v_j \in V_e}} \frac{1}{|V_e|}, \quad (12)$$

where $1/|V_e|$ normalises by hyperedge size following the homophily assumption (Tang et al., 2025a)—nodes sharing a small, tight hyperedge are more likely to encode similar knowledge than those sharing a large one—and $\min(\gamma(v_i), \gamma(v_j))$ ensures that high edge weights occur only when *both* nodes are useful. Structural embeddings are then obtained via L -step symmetric propagation following (Tang et al., 2025a):

$$\begin{aligned} \tilde{\mathbf{Z}} &= \mathbf{S} \mathbf{Z}, \\ \mathbf{S} &= (1 - \alpha)^L \hat{\mathbf{W}}^L + \alpha \sum_{l=0}^{L-1} (1 - \alpha)^l \hat{\mathbf{W}}^l, \quad (13) \\ \hat{\mathbf{W}} &= \hat{\mathbf{D}}^{-1/2} (\mathbf{W} + \mathbf{I}) \hat{\mathbf{D}}^{-1/2}, \end{aligned}$$

where $\mathbf{Z} \in \mathbb{R}^{|\mathcal{V}_s| \times d}$ stacks plain skill node embeddings, $\hat{\mathbf{D}}$ is the degree matrix of $(\mathbf{W} + \mathbf{I})$, $\alpha \in (0, 1)$ balances individual node identity against neighbourhood context, and L controls the propagation depth. Since node utility is already encoded in $\mathbf{W}$, the propagated embeddings $\tilde{\mathbf{Z}}$ jointly reflect structural co-occurrence and skill quality, and a pair (v_a, v_b) is selected as a merge candidate by similarity alone:

$$\text{sim}(\tilde{\mathbf{z}}_a, \tilde{\mathbf{z}}_b) \geq \delta_{\text{merge}}. \quad (14)$$

For each selected pair, the agent reasons over the contents of v_a, v_b , and their shared trajectory fragments to propose a merged node v_{new} that consolidates their knowledge into a higher-order skill. All hyperedge memberships are reassigned to v_{new} , preserving structural integrity across $\mathcal{G}$.

4 Experiments

4.1 Experiment Setup

Datasets and Benchmarks. We evaluate HYPERSKILL across three diverse benchmarks spanning different domains and capability requirements: (i) **xBench** (Chen et al., 2025), a benchmark assessing agentic planning, tool use, and multi-step

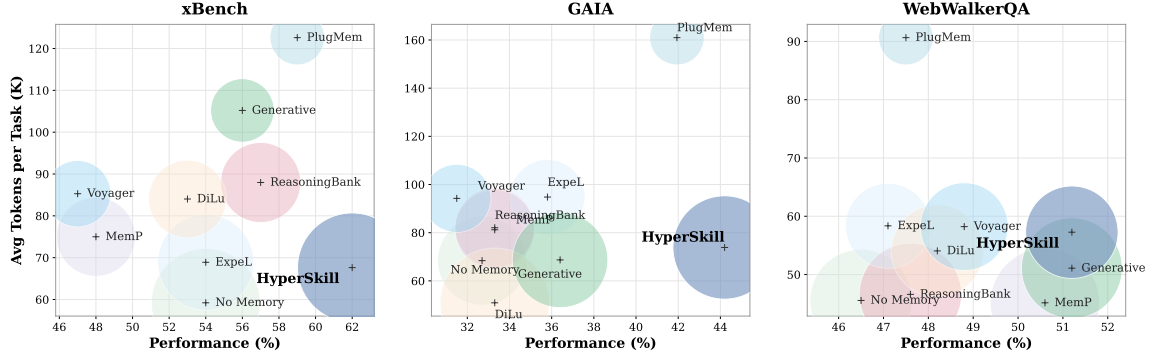


Figure 3: Performance vs. average tokens per task (K) across three benchmarks. Bubble size reflects the performance-to-token ratio.

reasoning across a broad range of real-world tasks; (ii) **GAIA** (Mialon et al., 2023), a set of real-world questions requiring multi-hop reasoning, multi-modality handling, web browsing, and tool-use proficiency; and (iii) **WebWalkerQA** (Wu et al., 2025a), a benchmark testing complex multi-turn web navigation across diverse real-world queries and webpages. Together, these benchmarks cover tool-augmented reasoning, open-ended web interaction, and compositional planning. More details are provided in Appendix A.

Baselines. We compare HYPERSKILL against ten representative memory systems spanning three categories. *Trajectory-level methods* that store raw or lightly processed trajectories: Voyager (Wang et al., 2023a), DILU (Wen et al., 2024), ExpeL (Zhao et al., 2024), and Generative (Shang et al., 2025). *Workflow- or insight-level methods* that extract and manage reusable knowledge units: AWM (Wang et al., 2025b), ReasoningBank (Ouyang et al., 2026), MemP (Fang et al., 2026), Cheatsheet (Suzgun et al., 2026), and MemEvolve (Zhang et al., 2025b). *Graph-based methods* that organize memory with relational structure: PlugMem (Yang et al., 2026). We also include a *No Memory* baseline that operates without any external memory module. All baselines use the same agent backbone and tool access for a fair comparison. More details are provided in Appendix A.

Implementation Details. We conduct all experiments with two backbone models: GPT-4o (Hurst et al., 2024) and Qwen3-30B-A3B (Yang et al., 2025), covering both proprietary and open-source settings. For HYPERSKILL, we use all-MiniLM-L6-v2 as the shared encoder $\phi(\cdot)$ for all node and hyperedge embeddings. A single retrieval budget governs subtask node retrieval, hyperedge retrieval, and the final skill ranking. Main-

tenance is triggered every $\lceil 0.1 \times N \rceil$ episodes, with pruning threshold $\tau_{\text{prune}}=0.2$ and minimum retrieval count $N_{\text{min}}=3$. The utility score blending coefficient is $\beta=0.7$. All hyperparameter and implementation details are provided in Appendix A.

4.2 Main Results

Table 1 reports success rate (SR), average steps, and average tool calls across xBench, GAIA, and WebWalkerQA under two backbone models. HYPERSKILL achieves the highest or success rate on nearly every benchmark-backbone combination. We highlight three observations. **① Consistent gains across benchmarks and backbones.** Under GPT-4o, HYPERSKILL outperforms all baselines on SR by +3.00 on xBench, +2.30 on GAIA, and +0.59 on WebWalkerQA relative to the second-best method. Under Qwen3-30B-A3B, the trend holds with gains of +6.00, +4.85, and +11.18 over No Memory, confirming that the improvements stem from the memory framework rather than backbone-specific behaviors. **② Improved SR without inflated execution cost.** A common concern with memory-augmented agents is that richer retrieved context may increase per-step overhead. HYPERSKILL largely avoids this: under GPT-4o on xBench, it achieves the highest SR while matching ExpeL for the fewest steps (5.27) and maintaining competitive tool calls (6.52). In contrast, methods such as ReasoningBank and Voyager require up to 8.22 steps and 12.27 calls on the same benchmark while delivering lower SR, indicating that their retrieved context introduces overhead without proportional benefit. **③ Naive memory accumulation can hurt.** Several baselines fall below No Memory on at least one setting: Cheatsheet drops -9.00 on xBench (Qwen3-30B-A3B), and Voyager drops -7.00 on xBench (GPT-4o). Without princi-

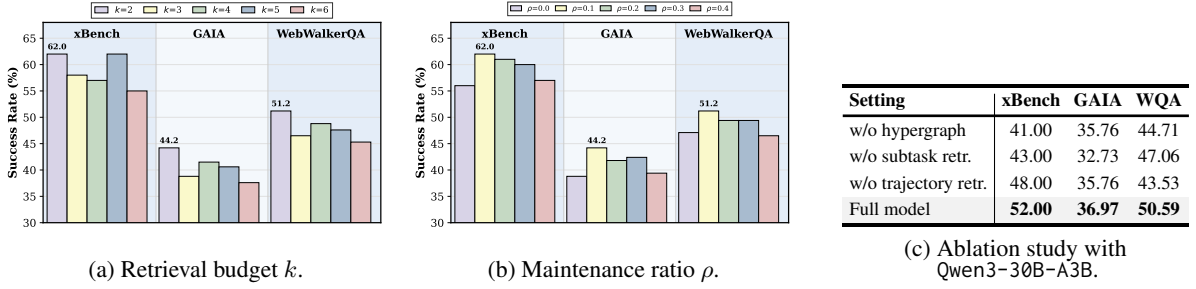


Figure 4: Sensitivity analysis and ablation study. (a, b) Success rate (%) on GPT-4o as we vary retrieval budget k and maintenance ratio ρ on xBench, GAIA, and WebWalkerQA. (c) Ablation of HYPERSKILL’s trajectory retrieval, subtask retrieval, and hypergraph structure.

pled maintenance, accumulated entries introduce noise that actively harms the agent. This underscores the importance of HYPERSKILL’s structure-aware evolution mechanism, which prunes low-utility nodes and merges redundant skills to keep memory compact and high-quality.

4.3 Analysis and Ablation Study

Cost Analysis. Figure 3 plots average tokens per task against performance, with bubble size proportional to the performance-to-token ratio. HYPERSKILL occupies the bottom-right region across all three benchmarks, achieving the highest performance at moderate token cost (67K, 72K, and 57K on xBench, GAIA, and WebWalkerQA, respectively). Methods such as Generative and Voyager consume over 100K tokens yet fall well below HYPERSKILL in performance. PlugMem, despite its knowledge-centric memory graph, incurs substantially higher token cost than HYPERSKILL while delivering lower performance. The gap stems from memory construction: PlugMem incurs separate LLM calls to extract state, subgoal, and reward annotations.

Sensitivity Analysis. We analyze the sensitivity of HYPERSKILL to two key hyperparameters: (i) retrieval budget k , which controls the number of candidates surfaced during both hyperedge and skill retrieval; and (ii) maintenance ratio ρ , which controls the fraction of low-utility entries pruned during each maintenance cycle. Figures 4(a–b) reveal two consistent patterns. First, performance favors smaller retrieval budgets: $k=2$ achieves the best results across all three benchmarks, with performance declining as k increases. This suggests that a compact retrieval set minimizes noise from lower-ranked candidates, and surfacing too many entries dilutes the signal from the most relevant skills. Second, the maintenance ratio peaks at $\rho=0.1$, with $\rho=0.2$ and $\rho=0.3$ performing similarly, indicating

that light pruning is sufficient to keep the memory effective. Disabling pruning entirely ($\rho=0$) lets low-utility entries accumulate and degrades retrieval quality, while overly aggressive pruning ($\rho=0.4$) removes still-useful skills.

Ablation Study. We ablate the two retrieval paths and the hypergraph structure, with results in Figure 4(c). *w/o trajectory retrieval* removes the path that matches the full task description against hyper-edge embeddings, relying solely on plan decomposition into subtask nodes and expansion through their incident hyperedges. Performance drops notably on WQA (50.59 $\rightarrow$ 43.53), suggesting that trajectory-level matching captures task-wide patterns that subtask expansion alone misses. *w/o subtask retrieval* removes the fine-grained path, collecting skills only from top- k task-matched hyperedges without plan decomposition or subtask-level expansion. This hurts xBench most and causes GAIA to drop to 32.73, indicating that subtask decomposition is essential for retrieving relevant skills on complex, multi-step tasks. *w/o hypergraph* replaces the entire dual-path pipeline with flat text-embedding retrieval over stored skills, removing all hypergraph structure. This yields the lowest results across all three benchmarks, confirming that the hypergraph’s n -ary co-occurrence relationships provide meaningful gains over a standard vector-store baseline. The full model combining both retrieval paths achieves the best results on all benchmarks, validating their complementarity.

4.4 Hypergraph Analysis

To illustrate how HYPERSKILL’s hypergraph organizes experiential knowledge, Figure 5 visualizes a representative subgraph of six trajectories from xBench. Subtask nodes are mostly trajectory-specific, whereas skill nodes are shared across hyperedges: *Cross-Source Validation* and *Targeted Database Search* each appear in multiple trajectory-

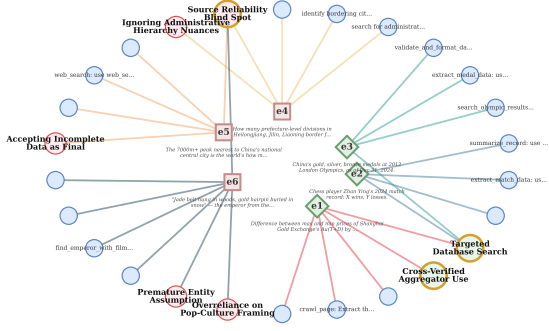


Figure 5: Hypergraph visualization on xBench. Trajectory hyperedges (◇ success, □ failure) connect skill nodes (○ success, ○ failure) and subtask nodes (●).

ries, enabling compositional reuse. Failed trajectories contribute error-pattern skills such as *Premature Entity Assumption* that cluster separately from positive strategies, allowing the agent to retrieve targeted warnings when similar subtask patterns recur. More examples are provided in Appendix B.

5 Conclusion

We presented HYPERSKILL, a hypergraph memory framework for self-evolving LLM agents. Each trajectory is stored as a hyperedge binding subtask decompositions and outcome-conditioned skills, preserving compositional structure that flat and pairwise designs discard. Dual-path retrieval surfaces relevant trajectories through both subtask-level and task-level matching, then ranks skills by co-occurrence frequency across retrieved hyperedges. Periodic maintenance prunes low-utility nodes and merges redundant skills via quality-weighted propagation, keeping memory compact as it grows. Experiments on xBench, GAIA, and WebWalkerQA with GPT-4o and Qwen3-30B-A3B-Instruct show consistent gains over ten baselines in success rate. Beyond accuracy, HYPERSKILL also attains the demonstrative performance-to-token ratio, reaching the highest success rate at a moderate token cost.

Limitations

HYPERSKILL relies on additional LLM calls for task decomposition, skill extraction, and merge decisions during memory maintenance, introducing extra inference cost. The current evaluation covers web navigation and multi-step reasoning benchmarks; generalization to others can be future work.

Finally, the effectiveness of hypergraph-based retrieval and maintenance depends on a sufficient number of accumulated trajectories, so HYPERSKILL may offer limited benefit in extremely low-data regimes where the memory graph is sparse.

References

- Petr Anokhin, Nikita Semenov, Artyom Sorokin, Dmitry Evseev, Andrey Kravchenko, Mikhail Burtsev, and Evgeny Burnaev. 2025. Arigraph: learning knowledge graph world models with episodic memory for llm agents. In *IJCAI*.
- Anthropic. 2024. [The claude 3 model family: Opus, sonnet, haiku](#).
- Kaiyuan Chen, Yixin Ren, Yang Liu, Xiaobo Hu, Haotong Tian, Tianbao Xie, Fangfu Liu, Haoye Zhang, Hongzhang Liu, Yuan Gong, and 1 others. 2025. xbench: Tracking agents productivity scaling with profession-aligned real-world evaluations. *arXiv preprint arXiv:2506.13651*.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*.
- Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. 2026. Memp: Exploring agent procedural memory. In *ACL Findings*.
- Huan-ang Gao, Jiayi Geng, Wenye Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Yiran Wu, and 1 others. 2025. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence. *arXiv preprint arXiv:2507.21046*.
- Alireza Ghafarollahi and Markus J. Buehler. 2025. Sciaagents: Automating scientific discovery through bio-inspired multi-agent intelligent graph reasoning. *Advanced Materials*.
- Izzeddin Gur, Hiroki Furuta, Austin V Huang, Mustafa Safdari, Yutaka Matsuo, Douglas Eck, and Aleksandra Faust. 2024. A real-world webagent with planning, long context understanding, and program synthesis. In *ICLR*.
- Bernal J Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. Hipporag: Neurobiologically inspired long-term memory for large language models. In *NeurIPS*.
- Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, and 1 others. 2025. Memory in the age of ai agents. *arXiv preprint arXiv:2512.13564*.

- Wei-Chieh Huang, Weizhi Zhang, Yueqing Liang, Yuanchen Bei, Yankai Chen, Tao Feng, Xinyu Pan, Zhen Tan, Yu Wang, Tianxin Wei, Shanglin Wu, Ruiyao Xu, Liangwei Yang, Rui Yang, Woosung Yang, Chin-Yuan Yeh, Hanrong Zhang, Haozhen Zhang, Siqi Zhu, and 41 others. 2026. A survey of agent memory in the second half: Towards self-evolving and long-horizon agents. *TMLR*.
- Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, and 1 others. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*.
- Dongming Jiang, Yi Li, Guanpeng Li, and Bingzhe Li. 2026. Magma: A multi-graph based agentic memory architecture for ai agents. In *ACL*.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.
- Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. Camel: Communicative agents for "mind" exploration of large language model society. *NeurIPS*.
- Bang Liu, Xinfeng Li, Jiayi Zhang, Jinlin Wang, Tanjin He, Sirui Hong, Hongzhang Liu, Shaokun Zhang, Kaitao Song, Kunlun Zhu, and 1 others. 2025a. Advances and challenges in foundation agents: From brain-inspired intelligence to evolutionary, collaborative, and safe systems. *arXiv preprint arXiv:2504.01990*.
- Yitao Liu, Chenglei Si, Karthik R Narasimhan, and Shunyu Yao. 2025b. [Contextual experience replay for self-improvement of language agents](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14179–14198, Vienna, Austria. Association for Computational Linguistics.
- David Lopez-Paz and Marc’Aurelio Ranzato. 2017. Gradient episodic memory for continual learning. In *NeurIPS*.
- Haoran Luo, Guanting Chen, Yandan Zheng, Xiaobao Wu, Yikai Guo, Qika Lin, Yu Feng, Zemin Kuang, Meina Song, Yifan Zhu, and 1 others. 2025. Hypergraphrag: Retrieval-augmented generation via hypergraph-structured knowledge representation. In *NeurIPS*.
- Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2023. Gaia: a benchmark for general ai assistants. In *ICLR*.
- Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. 2026. Reasoning-bank: Scaling agent self-evolving with reasoning memory. In *ICLR*.
- Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. 2023. Memgpt: towards llms as operating systems. *arXiv preprint arXiv:2305.00000*.
- Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: a temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*.
- Shuo Ren, Can Xie, Pu Jian, Zhenjiang Ren, Chunlin Leng, and Jiajun Zhang. 2025. Towards scientific intelligence: A survey of llm-based scientific agents. *arXiv preprint arXiv:2503.24047*.
- Yu Shang, Yu Li, Keyu Zhao, Likai Ma, Jiahe Liu, Fengli Xu, and Yong Li. 2025. Agentsquare: Automatic LLM agent search in modular design space. In *ICLR*.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *NeurIPS*.
- Haotian Sun, Yuchen Zhuang, Ling kai Kong, Bo Dai, and Chao Zhang. 2023. Adaplanner: Adaptive planning from feedback with language models. In *NeurIPS*.
- Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. 2026. [Dynamic cheat-sheet: Test-time learning with adaptive memory](#). In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7080–7106, Rabat, Morocco. Association for Computational Linguistics.
- Yashar Talebirad and Amirhossein Nadiri. 2023. Multi-agent collaboration: Harnessing the power of intelligent llm agents. *arXiv preprint arXiv:2306.03314*.
- Bohan Tang, Zexi Liu, Keyue Jiang, Siheng Chen, and Xiaowen Dong. 2025a. Training-free message passing for learning on hypergraphs. In *ICLR*.
- Xiangru Tang, Tianrui Qin, Tianhao Peng, Ziyang Zhou, Daniel Shao, Tingting Du, Xinming Wei, Peng Xia, Fang Wu, He Zhu, and 1 others. 2025b. Agent kb: Leveraging cross-domain experience for agentic problem solving. *arXiv preprint arXiv:2507.06229*.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023a. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv: Arxiv-2305.16291*.

- Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. 2023b. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *ACL*.
- Zhenhailong Wang, Haiyang Xu, Junyang Wang, Xi Zhang, Ming Yan, Ji Zhang, Fei Huang, and Heng Ji. 2025a. Mobile-agent-e: Self-evolving mobile assistant for complex tasks. *arXiv preprint arXiv:2501.11733*.
- Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025b. Agent workflow memory. In *ICML*.
- Tianxin Wei, Ting-Wei Li, Zhining Liu, Xuying Ning, Ze Yang, Jiaru Zou, Zhichen Zeng, Ruizhong Qiu, Xiao Lin, Dongqi Fu, and 1 others. 2026. Agentic reasoning for large language models. *arXiv preprint arXiv:2601.12538*.
- Licheng Wen, Daocheng Fu, Xin Li, Xinyu Cai, Tao Ma, Pinlong Cai, Min Dou, Botian Shi, Liang He, and Yu Qiao. 2024. Dilu: A knowledge-driven approach to autonomous driving with large language models. In *ICLR*.
- Jialong Wu, Wenbiao Yin, Yong Jiang, Zhenglin Wang, Zekun Xi, Runnan Fang, Linhai Zhang, Yulan He, Deyu Zhou, Pengjun Xie, and Fei Huang. 2025a. [WebWalker: Benchmarking LLMs in web traversal](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10290–10305, Vienna, Austria. Association for Computational Linguistics.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, and 1 others. 2024. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *COLM*.
- Rong Wu, Xiaoman Wang, Jianbiao Mei, Pinlong Cai, Daocheng Fu, Cheng Yang, Licheng Wen, Xueming Yang, Yufan Shen, Yuxin Wang, and 1 others. 2025b. Evolver: Self-evolving llm agents through an experience-driven lifecycle. *arXiv preprint arXiv:2510.16079*.
- Zhaofen Wu, Hanrong Zhang, Fulin Lin, Wujiang Xu, Xinran Xu, Yankai Chen, Henry Peng Zou, Shaowen Chen, Weizhi Zhang, Xue Liu, and 1 others. 2026. Gam: Hierarchical graph-based agentic memory for llm agents. *arXiv preprint arXiv:2604.12285*.
- Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, and 1 others. 2025. The rise and potential of large language model based agents: A survey. *Science China Information Sciences*.
- Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, and 1 others. 2026. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, and 1 others. 2024. Oworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *NeurIPS*.
- Ruiyao Xu and Kaize Ding. 2026. GNN-as-judge: Unleashing the power of LLMs for graph learning with GNN feedback. In *ICLR*.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2026. A-mem: Agentic memory for LLM agents. In *NeurIPS*.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. In *NeurIPS*.
- Ke Yang, Zixi Chen, Xuan He, Jize Jiang, Michel Galley, Chenglong Wang, Jianfeng Gao, Jiawei Han, and Chengxiang Zhai. 2026. Plugmem: A task-agnostic plugin memory module for llm agents. *arXiv preprint arXiv:2603.03296*.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. In *ICLR*.
- Juwei Yue, Chuanrui Hu, Jiawei Sheng, Zuyi Zhou, Wenyuan Zhang, Tingwen Liu, Li Guo, and Yafeng Deng. 2026. Hypermem: Hypergraph memory for long-term conversations. In *ACL*.
- Guibin Zhang, Muxin Fu, Guancheng Wan, Miao Yu, Kun Wang, and Shuicheng Yan. 2025a. G-memory: Tracing hierarchical memory for multi-agent systems. In *NeurIPS*.
- Guibin Zhang, Haotian Ren, Chong Zhan, Zhenhong Zhou, Junhao Wang, He Zhu, Wangchunshu Zhou, and Shuicheng Yan. 2025b. Memevolve: Meta-evolution of agent memory systems. *arXiv preprint arXiv:2512.18746*.
- Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. 2026. Memskill: Learning and evolving memory skills for self-evolving agents. *arXiv preprint arXiv:2602.02474*.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. Expel: Llm agents are experiential learners. In *AAAI*.

Boyuan Zheng, Michael Y Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and 1 others. 2025. Skillweaver: Web agents can self-improve by discovering and honing skills. *arXiv preprint arXiv:2504.07079*.

Longtao Zheng, Rundong Wang, Xinrun Wang, and Bo An. 2024. Synapse: Trajectory-as-exemplar prompting with memory for computer control. In *ICLR*.

Dengyong Zhou, Jiayuan Huang, and Bernhard Schölkopf. 2006. Learning with hypergraphs: Clustering, classification, and embedding. In *NeurIPS*.

Appendix Overview

- [Appendix A: Datasets and Implementation Details](#)
- [Appendix B: More Analysis](#)
- [Appendix C: Broader Impact](#)
- [Appendix D: Disclosure of LLM Use](#)
- [Appendix E: Extended Related Works](#)
- [Appendix F: Ethics Statement](#)
- [Appendix G: Additional Case Studies](#)
- [Appendix H: Algorithm](#)
- [Appendix I: Prompts](#)

A Datasets and Implementation Details

A.1 Dataset Details

We follow the evaluation protocol of MemEvolve (Zhang et al., 2025b) and adopt the same three benchmarks.

GAIA (Mialon et al., 2023) consists of 165 tasks organized into three difficulty levels: Level-1 (53 tasks), Level-2 (86 tasks), and Level-3 (26 tasks), requiring multi-hop reasoning, web browsing, and tool use.

WebWalkerQA (Wu et al., 2025a) covers complex multi-turn web navigation with 680 queries across four domains and over 1,373 webpages. We use the same 170-query subset as (Zhang et al., 2025b).

xBench (Chen et al., 2025) contains 100 tasks assessing agentic planning, tool use, and multi-step reasoning across diverse real-world scenarios.

A.2 Implementation Details

We build on the MemEvolve (Zhang et al., 2025b) codebase and adopt the same agent architecture across all methods for fair comparison. For all baselines, the memory retrieval count is set to 3 and text embeddings are computed using all-MiniLM-L6-v2 with cosine similarity as the retrieval metric.

LLM backends. For GPT-4o, we use the OpenAI API with temperature 0.7 and retrieval budgets $(k_u, k_e, k_s) = (2, 2, 2)$. For Qwen3-30B-A3B, we serve the model locally via vLLM (Kwon et al., 2023) on NVIDIA A100 80 GB GPUs with temperature 0.7 and retrieval budgets $(k_u, k_e, k_s) = (5, 5, 5)$. The sliding window of recent interaction history is set to $w = 3$ steps.

Memory update. The skill deduplication threshold is $\delta_{\text{dedup}} = 0.9$ and untested nodes receive a neutral prior $\gamma = 0.5$. The utility score blending coefficient is $\beta = 0.7$ (Eq. 3).

Maintenance. Maintenance is triggered every $\lceil 0.1 \times N \rceil$ tasks, where N is the total number of tasks in the benchmark, with pruning threshold $\tau_{\text{prune}} = 0.2$ and minimum retrieval count $N_{\text{min}} = 3$. The merge candidate threshold is $\delta_{\text{merge}} = 0.85$.

Reproducibility. Tasks are processed sequentially in a fixed random order (seed 42), shared across all methods.

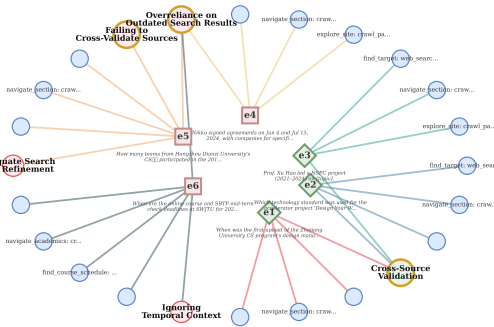


Figure 6: Hypergraph subgraph visualization on WebWalkerQA. Trajectory hyperedges ($\diamond$ success, $\square$ failure) connect skill nodes ($\circ$ success, $\square$ failure) and subtask nodes ($\bullet$).

B More Analysis

B.1 Self-Evolving Analysis

To examine whether HYPERSKILL’s hypergraph memory genuinely improves with experience, we plot the cumulative success rate as a function of episode index across all three benchmarks.

Figure 7 reveals three key findings. First, HYPERSKILL is the only method that exhibits a sustained upward trajectory: on GAIA, its cumulative accuracy climbs steadily from the early episodes through the final ones, confirming that the hypergraph memory accumulates transferable skills over time. Second, baselines with comparable final success rates, such as DiLu on xBench, reach their peak early and stagnate or dip in the middle episodes before recovering, indicating that their memory does not compound across tasks. Third, the self-evolving effect is more pronounced on benchmarks with greater task diversity (GAIA, WebWalkerQA) than on xBench, suggesting that HYPERSKILL benefits most when the hypergraph

can capture diverse cross-task skill co-occurrence patterns.

B.2 Latency Analysis

Beyond success rate, practical deployment requires acceptable wall-clock latency. Figure 8 reports average per-task execution time across all three benchmarks under both backbones.

Under Qwen3-30B-A3B, HYPERSKILL adds moderate overhead relative to the fastest baseline (No Memory): 42.6s vs. 26.5s on xBench, 33.6s vs. 33.6s on GAIA (matching DILU), and 31.3s vs. 23.9s on WebWalkerQA. Under GPT-4o, the pattern is similar: HYPERSKILL runs at 48.3s, 47.6s, and 55.3s, while the fastest baselines reach 29.7s, 32.5s, and 41.9s respectively. In both settings, the overhead stems primarily from the additional LLM call for task decomposition and skill extraction, not from hypergraph operations themselves. Combined with the token-efficiency results in Figure 3, these results confirm that HYPERSKILL achieves the best performance at a practical latency cost.

B.3 Memory Growth Analysis

Figure 9 shows how the hypergraph evolves over successive tasks. Subtask nodes grow roughly linearly since each task introduces a new decomposition, while both skill types grow sub-linearly: insertion-time deduplication prevents near-duplicate skills from entering the graph, and periodic structure-informed merging consolidates redundant entries, together keeping skill counts well below the number of hyperedges across all benchmarks and backbones.

B.4 Robustness Analysis

Ground-truth labels may be unavailable after each task in practice, yet following prior work (Fang et al., 2026; Zhang et al., 2025b), HYPERSKILL uses outcome signals to assign success/failure to extracted skills. We evaluate a *self-judge* variant in which the agent assesses its own trajectory outcome without access to the gold answer, and compare it against the *GT-judge* setting used in our main experiments. Table 2 reports results on GPT-4o. The self-judge variant retains 87.7–96.8% of the GT-judge performance across all three benchmarks, with the smallest gap on xBench (−2.00 SR) and the largest on GAIA (−5.37 SR). The modest degradation suggests that HYPERSKILL is robust to noisy outcome signals: co-occurrence ranking over multiple retrieved hyperedges dilutes

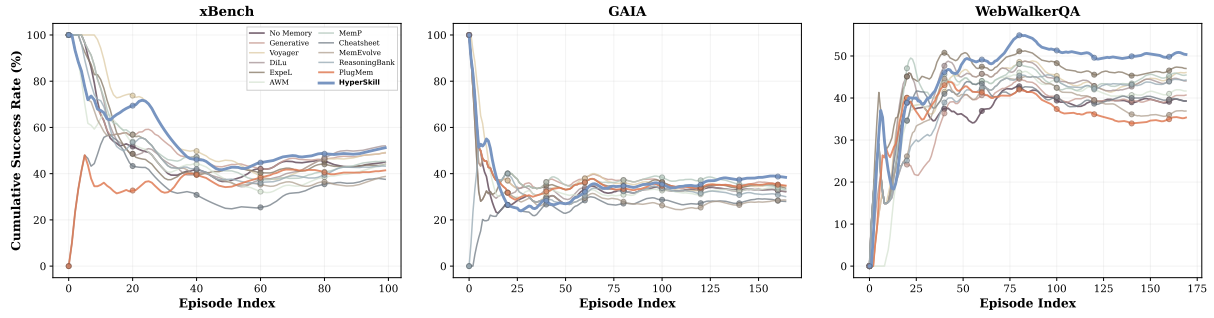


Figure 7: Cumulative success rate (%) over episode index on Qwen3-30B-A3B. HYPERSKILL exhibits an upward trend across benchmarks.

Table 2: Comparison of GT-judge and self-judge on GPT-4o. Retention = Self-judge SR / GT-judge SR $\times$ 100%.

Benchmark	GT-judge	Self-judge	Retention
xBench	62.00	60.00	96.8%
GAIA	44.24	38.87	87.7%
WebWalkerQA	51.18	47.33	92.4%

the effect of occasional mislabeled skills, allowing the structural signal to remain reliable even without ground-truth feedback.

B.5 Hypergraph Visualization

Figure 6 shows the WebWalkerQA counterpart of the xBench hypergraph subgraph in Figure 5.

C Broader Impact

HYPERSKILL is a general-purpose memory framework for LLM agents and does not target any specific high-risk application. By enabling agents to learn from past failures, it may reduce redundant computation and improve efficiency in deployed systems. However, as with any self-evolving system, accumulated memory could reinforce biases present in early trajectories if not periodically audited. We encourage practitioners to monitor stored skills for unintended patterns.

D Disclosure of LLM Use

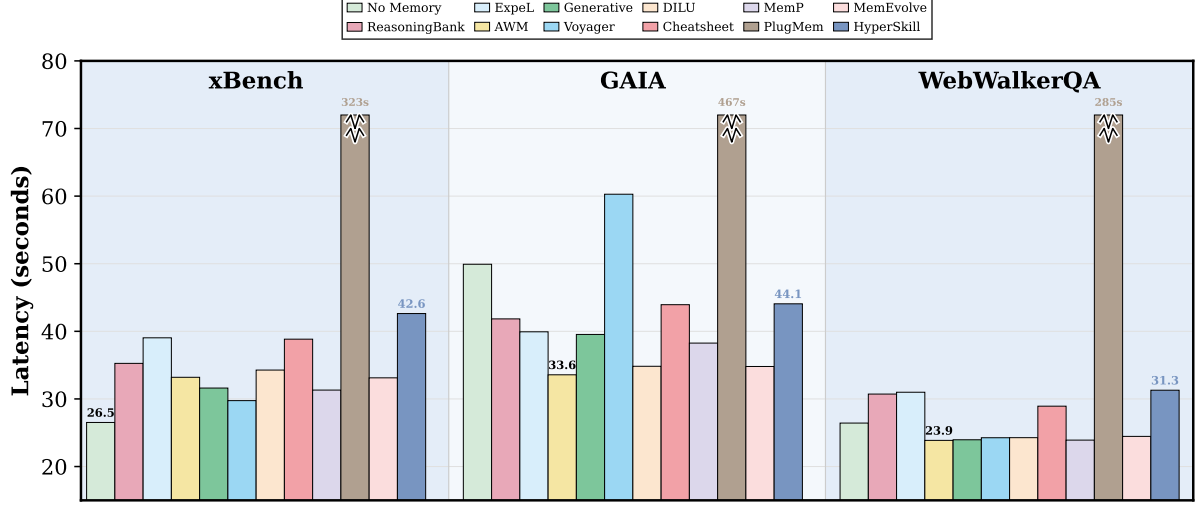
During the preparation of this manuscript, LLM-based tools were used to assist with grammar correction, code debugging, and figure design. All scientific content, experimental design, and analysis were conducted by the authors, who take full responsibility for the final manuscript.

E Extended Related Works

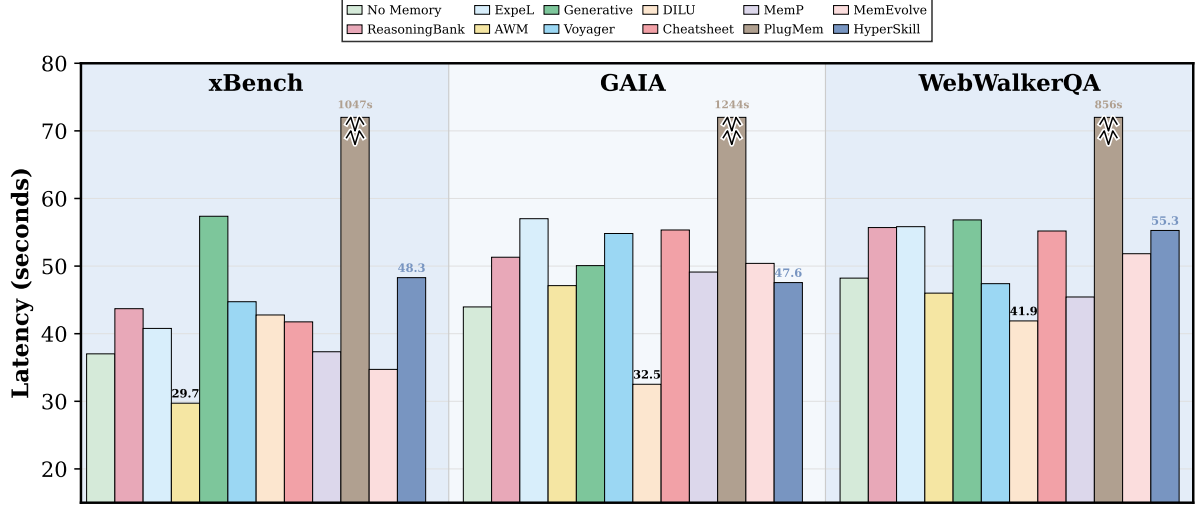
LLM Agent Systems. Large language models are increasingly being deployed as autonomous agents that interact with complex environments through multi-step reasoning and actions, moving beyond traditional one-shot chatbot paradigms (Liu et al., 2025a; Xi et al., 2025; Wei et al., 2026). A central challenge is enabling agents to decompose complex tasks into manageable sub-problems and make sequential decisions. ReAct (Yao et al., 2022) interleaves chain-of-thought reasoning with environment actions, establishing a foundational paradigm for single-agent planning. Subsequent work improves task decomposition through hierarchical planning (Wang et al., 2023b; Sun et al., 2023) and enhances decision quality via self-reflection (Shinn et al., 2023). Multi-agent systems (Li et al., 2023; Wu et al., 2024; Talebirad and Nadiri, 2023) extend this paradigm further by assigning specialized roles to multiple agents that communicate and coordinate, enabling collaborative problem-solving on tasks beyond the scope of a single agent. While these advances have significantly broadened the capability frontier of LLM agents, most systems treat each task episode in isolation, limiting their ability to improve over time on recurring or compositionally related task patterns.

Memory Mechanisms in Agent Systems. We organize existing experiential memory systems along three dimensions: *what is stored*, *how memory is structured and retrieved*, and *how memory evolves*. Table 3 summarizes the comparison.

On what is stored, early systems retain raw trajectories or extract insights from each trajectory (Zhao et al., 2024; Wen et al., 2024; Wang et al., 2023a). Subsequent work abstracts experience into more compact forms: workflows (Wang et al., 2025b), reasoning strategies (Ouyang et al.,



(a) Qwen3-30B-A3B.



(b) GPT-4o.

Figure 8: Average per-task wall-clock latency (seconds) across xBench, GAIA, and WebWalkerQA. Lower is better. Annotated bars mark the fastest baseline in each panel and HYPERSKILL’s latency for reference.

2026), executable API skills (Zheng et al., 2025), or tips and shortcuts (Suzgun et al., 2026; Wang et al., 2025a). Recent training-based approaches go further: SkillIRL (Xia et al., 2026) distills trajectories into a hierarchical skill library and co-evolves it with the agent’s policy via reinforcement learning (GRPO), while MemSkill (Zhang et al., 2026) treats memory operations themselves as learnable skills whose selection is optimized through RL and whose skill bank evolves by analyzing hard cases. However, all of these systems store knowledge units in isolation, discarding the compositional relationships among subtasks and skills within each trajectory. HYPERSKILL is the only system that preserves this joint compositional structure.

On how memory is structured and retrieved, the majority of systems use flat vector stores with semantic retrieval (Wang et al., 2023a; Wen et al., 2024; Shang et al., 2025), while a few adopt contrastive retrieval that compares successes and failures (Zhao et al., 2024; Ouyang et al., 2026). AWM (Wang et al., 2025b) concatenates all entries into the prompt, and SkillWeaver (Zheng et al., 2025) uses function signature matching. Graph-based systems such as G-Memory (Zhang et al., 2025a) and Agent-KB (Tang et al., 2025b) introduce pairwise edges but decompose the n -ary trajectory context into disconnected binary links. HYPERSKILL uses hyperedges to preserve the full trajectory context and retrieves via a dual-path mech-

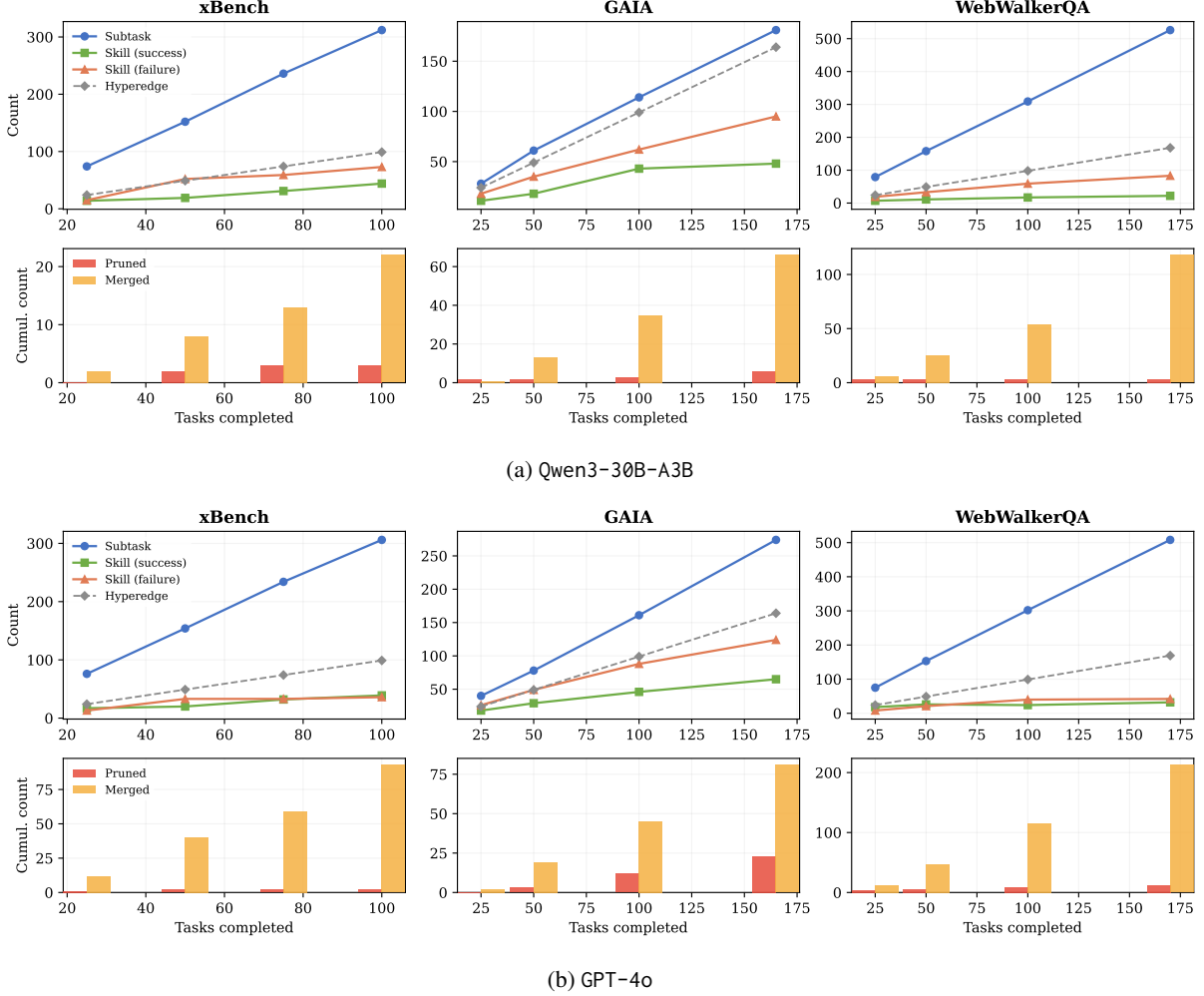


Figure 9: Hypergraph memory growth (top row) and cumulative maintenance operations (bottom row) across benchmarks.

anism that combines subtask-level and task-level queries.

On how memory evolves, most systems accumulate knowledge without any curation (Wang et al., 2023a; Zhao et al., 2024; Wen et al., 2024; Ouyang et al., 2026; Wang et al., 2025b). The few that do maintain memory apply lightweight operations such as test-and-prune (Zheng et al., 2025), episodic consolidation (Zhang et al., 2025a), deduplication (Tang et al., 2025b), or failure-driven adjustment (Fang et al., 2026), each applied independently per node without considering memory topology. SkillRL (Xia et al., 2026) and MemSkill (Zhang et al., 2026) evolve their skill libraries through RL training loops, but this evolution operates on model weights rather than on an explicit memory structure. HYPERSKILL performs structure-aware maintenance that jointly considers node utility and structural proximity via quality-weighted hypergraph propagation.

We also compare graph-structured memory systems specifically in Table 4. Most graph-based approaches target factual memory, recording environmental or user knowledge in knowledge graphs (Gutiérrez et al., 2024; Chhikara et al., 2025), temporal graphs (Rasmussen et al., 2025), hierarchical graphs (Wu et al., 2026), or multi-view graphs (Jiang et al., 2026). HyperMem (Yue et al., 2026) uses a three-level hypergraph (topics, episodes, facts) with coarse-to-fine retrieval, but targets factual conversational memory rather than experiential task knowledge and does not perform structure-aware maintenance. Among experiential memory systems, PlugMem (Yang et al., 2026) and G-Memory (Zhang et al., 2025a) use pairwise edges that lose the joint trajectory context, and Agent-KB (Tang et al., 2025b) combines lexical and semantic indices without explicit graph structure over skills. HYPERSKILL is the first experiential memory system to use n -ary hyper-

Table 3: Taxonomy of experiential memory systems for LLM agents. Traj. = raw trajectories; Strat. = reasoning strategies; Tool Lib. = callable function registries; Skill Lib. = hierarchical skill libraries. ‡ Training-based methods that internalize skills into model weights via reinforcement learning.

Method	What is Stored		How Structured & Retrieved		How Evolved
	Knowledge Form	Compositional?	Structure	Retrieval	Maintenance
Voyager (Wang et al., 2023a)	Traj. & Tips	✗	Vector DB	Semantic	None
ExpeL (Zhao et al., 2024)	Traj. & Insights	✗	Vector DB	Contrastive	None
Generative (Shang et al., 2025)	Traj. & Insights	✗	Vector DB	Semantic	None
DILU (Wen et al., 2024)	Traj.	✗	Vector DB	Semantic	None
AWM (Wang et al., 2025b)	Workflows	✗	In-Prompt	All-in-Prompt	None
Mobile-E (Wang et al., 2025a)	Tips & Shortcuts	✗	Vector DB	Semantic	None
Cheatsheet (Suzgun et al., 2026)	Tips & Shortcuts	✗	JSON	Semantic	None
ReasoningBank (Ouyang et al., 2026)	Reasoning Strat.	✗	Vector DB	Contrastive	None
SkillWeaver (Zheng et al., 2025)	APIs	✗	Tool Lib.	Function Match	Test & Prune
G-Memory (Zhang et al., 2025a)	Tips & Workflows	✗	Graph	Graph + Semantic	Consolidation
Agent-KB (Tang et al., 2025b)	Tips & Workflows	✗	Hybrid DB	Hybrid	Deduplication
Memp (Fang et al., 2026)	Tips & Workflows	✗	JSON	Semantic	Failure-driven
EvolveR (Wu et al., 2025b)	Tips & Workflows	✗	JSON	Contrastive	Update & Pruning
SkillRL [‡] (Xia et al., 2026)	Hierarchical Skills	✗	Markdown	Adaptive	RL Co-Evolution
MemSkill [‡] (Zhang et al., 2026)	Learnable Skills	✗	Markdown	RL-guided	Skill Evolution
HYPERSKILL	Workflows & Skills	✓	Hypergraph	Dual-path	Quality + Topology

Table 4: Graph-structured memory systems for LLM agents. “Memory Type” categorizes the primary function: *Factual* records environmental or user knowledge; *Experiential* accumulates reusable task knowledge from agent trajectories. “Edge Type” distinguishes *Pairwise* (binary) from *n-ary* (hyperedges connecting arbitrary node sets). † Agent-KB shares knowledge across frameworks but executes with a single agent.

Method	Memory Scope		Graph Structure			Evolution
	Mem. Type	MAS	Graph Type	Edge	Retrieval	Maintenance
HippoRAG (Gutiérrez et al., 2024)	Factual	✗	KG	Pairwise	PPR Activation	None
AriGraph (Anokhin et al., 2025)	Factual	✗	KG + Episodic	Pairwise	Semantic + Episodic	Graph Update
A-Mem (Xu et al., 2026)	Factual	✗	Zettelkasten	Pairwise	Semantic + Linking	Self-Organizing
Mem0g (Chhikara et al., 2025)	Factual	✗	KG + Vector	Pairwise	Hybrid	Deduplication
Zep (Rasmussen et al., 2025)	Factual	✗	Temporal KG	Pairwise	Temporal + Semantic	Temporal Inval.
GAM (Wu et al., 2026)	Factual	✗	Hierarchical	Pairwise	Multi-factor Trav.	Event-driven Consol.
MAGMA (Jiang et al., 2026)	Factual	✗	Multi-Graph	Pairwise	Policy-guided Trav.	Async. Consol.
HyperMem (Yue et al., 2026)	Factual	✗	Hypergraph	<i>n</i> -ary	Coarse-to-fine	None
HyperGraphRAG (Luo et al., 2025)	Factual	✗	Hypergraph	<i>n</i> -ary	Hypergraph Retrieval	None
PlugMem (Yang et al., 2026)	Experiential	✗	Dual KG	Pairwise	Subgraph Retrieval	None
G-Memory (Zhang et al., 2025a)	Experiential	✓	Hierarchical	Pairwise	Graph + Semantic	Episodic Consol.
Agent-KB [†] (Tang et al., 2025b)	Experiential	✗	Hybrid	Pairwise	Hybrid	Deduplication
HYPERSKILL	Experiential	✗	Hypergraph	<i>n</i> -ary	Dual-path + κ	Quality + Topology

edges, enabling co-occurrence-based skill ranking and topology-aware maintenance that pairwise designs cannot support.

F Ethics Statement

We provide comprehensive methodological details, including hyperparameters, prompts, and algorithm pseudocode, to enable full reproducibility. Our framework is evaluated on publicly available benchmarks (xBench, GAIA, WebWalkerQA) and does not collect or process personal information. LLM-based tools were used solely for grammar correction, code debugging, and figure design; all scientific content, experimental design, and analysis

were conducted by the authors. While HyperSkill is designed as a general-purpose memory framework for benign research, we acknowledge that accumulated memory could reinforce biases present in early trajectories if not periodically audited, and we encourage practitioners to monitor stored skills for unintended patterns.

G Additional Case Studies

To illustrate how HYPERSKILL’s retrieval and evolution mechanisms operate in practice, we walk through a representative GAIA task and a snapshot of memory maintenance in Figure 10. We also present three additional end-to-end case studies spanning the three benchmarks. Each case reports the task, the decomposition used to drive fine-grained retrieval, the skills and mistakes surfaced from the hypergraph, the agent’s step-wise execution trace, and the final outcome. Together they illustrate how HYPERSKILL’s dual-path retrieval translates into concrete action improvements across heterogeneous domains.

Case 1: xBench — Literary Trivia

Task (translated from Chinese): A 19th-century French novel caused a morality scandal and lawsuit upon publication. In the 1991 film adaptation, what flavor of prop poison did the lead actress insist on using for the suicide scene?

Golden answer: bitter-almond flavor.

Plan Decomposition

1. `search_literary_work`: identify the 19th-century French novel via scandal keywords.
2. `find_movie_adaptations`: locate the 1991 film adaptation and lead actress.
3. `verify_prop_taste`: crawl film trivia sources for the specific flavor detail.
4. `confirm_taste_detail`: validate the exact flavor description.

Retrieved Memory

Past experience (distilled lessons):

- ✓ *When verifying historical details, prioritize authoritative sources and cross-verify claims across multiple trusted references.*
- ✗ *Never assume that detailed data is available in public summaries; always use precise, entity-focused queries to locate authoritative sources.*

Skills:

- **Multi-Stage Verification:** break complex cross-domain questions into sequential verification steps.
- **Direct Source Confirmation:** prioritize official reference pages over aggregator sites.
- **Source Deep Dive:** use page crawling to extract precise details rather than relying on snippets.

Mistakes to avoid:

- **Over-Reliance on Web Search:** repeatedly issuing general searches without verifying authoritative sources.

Agent Execution (10 steps)

- 1–2 `web_search` → identifies *Madame Bovary* by Gustave Flaubert; initial poison query.
- 3–5 Confirms 1991 film by Claude Chabrol with Isabelle Huppert as Emma Bovary.
- 6–8 Progressive refinement: searches behind-the-scenes trivia across multiple sources.
- 9–10 Final verification: extracts bitter-almond flavor detail from an authoritative film database.

Outcome

✓ **SUCCESS** — the lead actress insisted on using a bitter-almond-flavored prop poison to authentically replicate the taste of cyanide.

How memory helped: The *Multi-Stage Verification* skill guided decomposition of a cross-domain chain (literature → film → behind-the-scenes trivia), while the *Over-Reliance on Web Search* error pattern prevented looping on generic queries. The retrieved lesson about cross-verifying across trusted references reinforced the agent’s move from aggregator snippets to authoritative film databases.

Case 2: GAIA — Scientific Calculation

Task: What integer-rounded percentage of the total length of the harlequin shrimp recorded in Omar Valencia-Mendez’s 2017 paper was the sea star fed to the same type of shrimp in G. Curt Fiedler’s 2002 paper?

Golden answer: 22.

Plan Decomposition

1. `inspect_file_as_text`: read the attached file for data from both papers.
2. `python_code`: calculate the integer-rounded percentage.

Retrieved Memory

Past experience (distilled lessons):

- ✓ *When seeking precise values from academic papers, prioritize direct access to the original source through targeted searches with author names and publication years.*
- ✗ *Never retry a tool call without diagnosing system-level errors first; always verify infrastructure readiness before re-attempting.*

Skills:

- **Domain-Specific Query Refinement:** include domain-specific terms and reliable source filters in queries for technical data.
- **Unit Conversion:** convert units consistently before performing calculations.

Mistakes to avoid:

- **Inadequate Context Refinement:** failing to break tasks into verifiable sub-questions leads to unstructured search.

Agent Execution (7 steps)

- 1–2 Parallel `web_search` for both papers; `crawl_page` on Fiedler 2002 via Oxford Academic.
- 3–4 Refines query with specific measurements; reads Valencia-Mendez 2017 PDF directly.
- 5–6 Cross-references: shrimp total length = 4.5 cm, sea-star piece = 1 cm.
- 7 `final_answer`: $\text{round}(1/4.5 \times 100) = 22\%$.

Outcome

✓ **SUCCESS** — the sea star was approximately 22% of the harlequin shrimp’s total length.

How memory helped: *Domain-Specific Query Refinement* pushed the agent to use precise scientific terms (“total length”, “harlequin shrimp”) rather than generic queries. The *Inadequate Context Refinement* mistake warned against unstructured searching, guiding the agent to split the task into two parallel data-retrieval subtasks (one per paper) before computing the ratio.

Case 3: WebWalkerQA — Game Knowledge

Task (translated from Chinese): In *Age of Empires IV*, what are the strategic differences between the Malian and Chinese civilizations in their respective fourth age?

Golden answer: the Malians leverage the Griot Bara landmark for global buffs, while the Chinese rely on the Great Wall as a defensive advantage.

Plan Decomposition

1. explore_site: crawl ageofempires.com homepage for site structure.
2. navigate_section: crawl the civilizations section for Malian and Chinese details.
3. find_target: search with domain-specific keywords on the official site.

Retrieved Memory

Past experience (distilled lessons):

- ✓ *When institutional websites fail to surface specific records, precise multi-source searches are essential for retrieving accurate information.*
- ✗ *Relying on automated crawling without validating access or adapting to site-specific barriers leads to failed data retrieval.*
- ✗ *Starting with external searches before fully navigating internal websites risks missing structured and unindexed information.*

Skills:

- **Internal First Strategy:** thoroughly explore internal site structure before resorting to external search tools.
- **Specific Query Refinement:** use precise, domain-specific keywords when searching internal or external tools.
- **Verify with Multimodal Sources:** when internal search yields limited results, use external sources to gather complementary information.

Mistakes to avoid:

- **Premature External Search:** relying on external web search before thoroughly exploring internal site navigation.
- **Incomplete Internal Crawling:** failing to systematically access relevant sections of a website.

Agent Execution (11 steps)

- 1–2 `crawl_page(ageofempires.com)` → explores homepage; `web_search` for general info.
- 3–5 Iterative crawling of official-site civilization sections.
- 6–8 Consults community sources (Reddit, YouTube) for fourth-age specifics.
- 9–11 `crawl_page` on the Fandom wiki for Malians and Chinese → extracts full details.

Outcome

✓ **SUCCESS** — detailed comparison: the Malians favor economic guerrilla tactics anchored by the Griot Bara landmark; the Chinese rely on fast building, Great Wall defense, and chemical technology.

How memory helped: The *Internal First Strategy* skill prevented the agent from jumping to external searches immediately, instead exploring the official site structure first. Three failure lessons all reinforced the same pattern: navigate internally before going external. This is a pattern learned across 6+ previous WebWalkerQA episodes involving institutional websites, now successfully transferred to a gaming wiki.

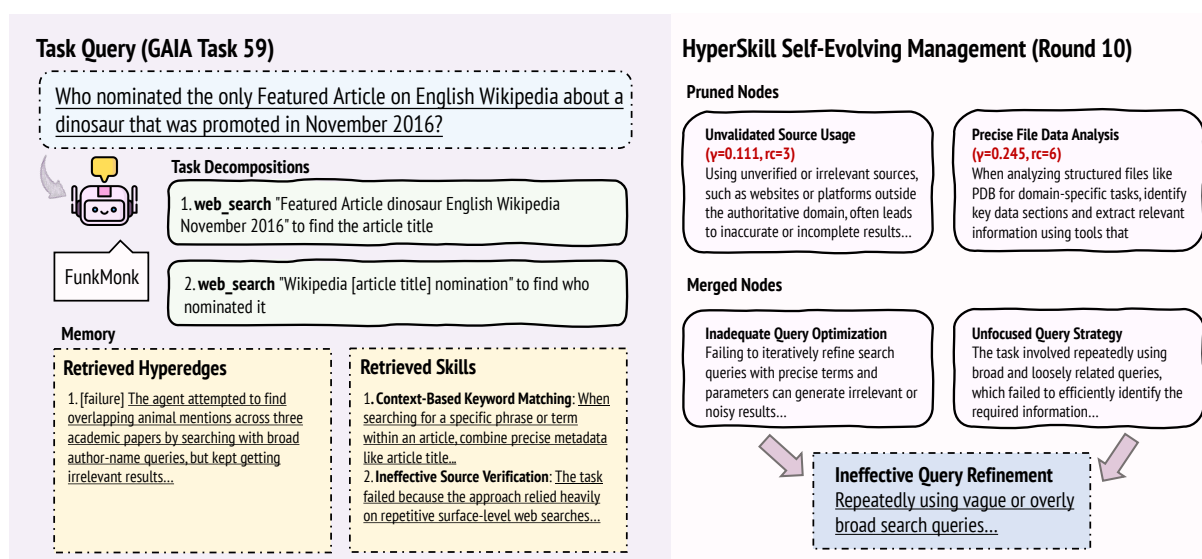


Figure 10: Case study on GAIA Task 59.

H Algorithm

Algorithm 1 summarizes the full HYPERSKILL pipeline per task, and Algorithm 2 describes the periodic memory evolution.

Algorithm 1 HYPERSKILL: Per-Task Pipeline

Require: Task d_q ; memory $\mathcal{G} = (\mathcal{V}, \mathcal{E})$; budgets k_u, k_e, k_s ; dedup threshold δ_{dedup}

Ensure: Updated memory $\mathcal{G}$

```

    % Dual-path hyperedge retrieval (§3.3)
    1:  $\mathcal{P}_0 \leftarrow \text{LLM-DECOMPOSE}(d_q)$  ▷ Subtask decomposition
    2:  $R_u \leftarrow \text{top-}k_u \text{ subtask nodes by } \text{sim}(\phi(\mathcal{P}_0), \phi(c_v))$  ▷ Eq. 4
    3:  $\mathcal{E}_{\text{sub}} \leftarrow \{e \in \mathcal{E} \mid V_e \cap R_u \neq \emptyset\}$  ▷ Subtask path
    4:  $\mathcal{E}_{\text{traj}} \leftarrow \text{top-}k_e \text{ hyperedges by } \text{sim}(\phi(d_q), \mathbf{h}_e)$  ▷ Trajectory path ▷ Eq. 5
    5:  $\mathcal{E}^* \leftarrow \mathcal{E}_{\text{sub}} \cup \mathcal{E}_{\text{traj}}$  ▷ Fuse ▷ Eq. 6
    % Skill ranking & execution (§3.4)
    6:  $R_s \leftarrow \bigcup_{e \in \mathcal{E}^*} V_e^s$ ;  $\kappa(v) \leftarrow |\{e \in \mathcal{E}^* \mid v \in V_e\}|$  for each  $v \in R_s$  ▷ Eq. 8
    7:  $\mathcal{S}_q \leftarrow \text{top-}k_s \text{ skills by } \kappa(v)$ ;  $\mathcal{L}_q \leftarrow \{\ell_e \mid e \in \mathcal{E}^*\}$  ▷ Eq. 9
    8: Execute task with context  $(\mathcal{S}_q, \mathcal{L}_q)$ ; observe outcome  $r_i$ , steps  $T_i$  ▷ Eq. 10
    % Memory update
    9: Extract skills  $S_i^{\text{new}}$  and lesson  $\ell_{e_i}$  from trajectory  $\tau_i$ 
    10: Deduplicate: merge into existing skill if  $\text{sim} \geq \delta_{\text{dedup}}$ , else add new node
    11: Add hyperedge  $e_{\text{new}} = (\mathcal{P}_0 \cup S_i^{\text{new}}, d_q, \ell_{e_i})$  to  $\mathcal{E}$ 
    12: Update utility  $\gamma(\cdot)$  via Eq. 3 for all retrieved elements

```

Algorithm 2 HYPERSKILL: Periodic Memory Evolution (every $\lceil 0.1 \times N \rceil$ tasks)

Require: Memory $\mathcal{G} = (\mathcal{V}, \mathcal{E})$; thresholds $\tau_{\text{prune}}, N_{\text{min}}, \delta_{\text{merge}}$; propagation depth L

```

    % Quality-driven pruning (§3.5)
    1: Remove all  $v$  with  $\nu(v) \geq N_{\text{min}}$  and  $\gamma(v) < \tau_{\text{prune}}$  from  $\mathcal{V}$  and incident hyperedges ▷ Eq. 11
    % Structure-informed merging (skill nodes  $\mathcal{V}_s$  only)
    2:  $W_{ij} \leftarrow \min(\gamma(v_i), \gamma(v_j)) \cdot \sum_{v_i \in V_e, v_j \in V_e} |V_e|^{-1}$  ▷ Quality-weighted co-occurrence ▷ Eq. 12
    3:  $\tilde{\mathbf{Z}} \leftarrow \mathbf{S} \mathbf{Z}$  via  $L$ -step propagation ▷ Structural embeddings ▷ Eq. 13
    4: for each pair  $(v_a, v_b)$  with  $\text{sim}(\tilde{\mathbf{z}}_a, \tilde{\mathbf{z}}_b) \geq \delta_{\text{merge}}$  do
    5:    $v_{\text{new}} \leftarrow \text{LLM-MERGE}(v_a, v_b)$ ; reassign hyperedge memberships ▷ Eq. 14
    6: end for

```

I Prompts

Task Decomposition Prompt (GAIA)

Decompose this task into 1–3 SHORT steps. Minimize steps to save context window.

Available tools:

- `web_search(query)`: Search with SPECIFIC keywords (names, dates, exact phrases)
- `crawl_page(url)`: Read a URL — use ONLY when you need specific page content
- `inspect_file_as_text(path)`: Read attached files — START HERE if task has a file
- `inspect_file_as_image(path)`: View images/figures
- `python code`: Calculations, counting, data processing

Key principles:

- If a file is attached, read it FIRST
- Use `web_search` with the MOST specific terms possible
- Prefer python code for calculations over searching

Task: {query}

Return ONLY a JSON array: [{"name": "action_name", "description": "use <tool> to <specific action>"}]

Task Decomposition Prompt (WebWalkerQA)

Decompose this web navigation task into 2–4 steps.

Available tools:

- `web_search(query)`: Search for specific pages on the website
- `crawl_page(url)`: Read content from a URL

Navigation strategy: ALWAYS start by crawling the root URL to understand site structure. Then navigate to the relevant section (News, Awards, Faculty, etc.) before searching for specifics.

Example for “Find what award Prof X won in 2021 on university.edu”:

```
[{"name": "explore_site",
  "description": "crawl_page(root_url)
  to find navigation menu"},
{"name": "navigate_section",
  "description":
  "crawl_page(root_url/awards)"},
{"name": "find_target",
  "description":
  "web_search('Prof X award 2021
  site:university.edu')"}]
```

Task: {query}

Return ONLY a JSON array.

Task Decomposition Prompt (xBench)

Decompose this data task into 2–4 steps. Each step must specify WHICH DATA to get and HOW to process it.

Available tools:

- `web_search(query)`: Search for data sources
- `crawl_page(url)`: Read data from specific URLs
- `python code`: Run calculations, unit conversions, comparisons

Task: {query}

Return ONLY a JSON array: [{"name": "action_name", "description": "use <tool> to <specific data action>"}]

Skill Extraction Prompt (Success)

Extract reusable knowledge from this SUCCESSFUL task.

CRITICAL RULES:

- Names must be SHORT reusable pattern names (3–5 words), NOT task-specific. Bad: “Search for Zip Codes”. Good: “Targeted Database Search”.
- Content must be a COMPLETE paragraph (2–4 sentences) that covers: WHAT to do, WHEN to apply it, and WHY it works.
- Extract 1–3 items max. Quality over quantity.
- Do NOT include task-specific details (exact names, IDs, URLs). Make knowledge transferable.

Task: {query}

Trajectory: {trajectory}

Result: {result}

Extract skills (what strategies worked well).

Extract as JSON:

```
{
  "skills": [
    {
      "name": "Short Skill Name",
      "content":
        "2-4 sentence paragraph..."
    },
    {
      "name": "Short Skill Name",
      "content":
        "2-4 sentence paragraph..."
    }
  ],
  "knowledge_fragment":
    "ONE OR TWO sentences stating the
    core transferable lesson."
}
```

Mistake Extraction Prompt (Failure)

Extract reusable lessons from this FAILED task.

CRITICAL RULES: (same as success extraction)

Task: {query}

Trajectory: {trajectory}

Result: {result}

Extract mistakes (what went wrong and how to avoid it).

Extract as JSON:

```
{
  "mistakes": [
    {
      "name": "Short Mistake Name",
      "content":
        "2-4 sentence paragraph
        describing what fails, why, and
        what to do instead."
    },
    {
      "name": "Short Mistake Name",
      "content":
        "2-4 sentence paragraph
        describing what fails, why, and
        what to do instead."
    }
  ],
  "knowledge_fragment":
    "ONE OR TWO sentences
    stating the core failure mode to avoid."
}
```

Structure-Informed Merging Prompt

These two {type}s are structurally and semantically related. Consolidate them into ONE higher-order {type} that captures the knowledge from both.

{TYPE} A:

Name: {name_a}

Content: {content_a}

{TYPE} B:

Name: {name_b}

Content: {content_b}

Create a consolidated {type} that:

- Has a SHORT reusable name (3–5 words)
- Has content (2–4 sentences) covering BOTH strategies/patterns
- Is more general and widely applicable than either individual {type}

Reply with ONLY JSON: {"name": "...", "content": "..."}.

Retrieved Memory Format (Agent Context)

The following memory is assembled once per episode and provided to the agent at the beginning of execution:

Past Experience

(Distilled lessons from top-k retrieved hyperedges)

1. [success] When official exchange data is rate-limited, cross-verified third-party aggregators provide a reliable fallback...
2. [failure] Relying on generic search terms instead of querying domain-specific databases yields incomplete data...

Relevant Skills

1. **Targeted Database Search:** Search for specific financial metrics using precise query terms that include the asset, time frame, and required data points...

Mistakes to Avoid

1. **Overreliance on Unverified Aggregators:** Using third-party summaries or social media posts as primary sources for quantitative claims leads to incorrect results...